Deep Learning for finance:

Neural Network application for sentiment analysis of economic texts

Ferraresi Gabriele (736903), Lecchi Alessia (731917), Pagani Giacomo (737294)

1. Introduction

This project is aimed at performing Financial Sentiment Analysis using Machine Learning and Deep Learning techniques.

Financial sentiment analysis consists of automatically identifying the polarity (positive, negative, or neutral) expressed in financial texts such as news articles, earnings reports, or corporate communications.

The applied methodology combines classical Natural Language Processing (NLP) approaches, such as Bag-of-Words and TF-IDF representations, with supervised learning algorithms including Logistic Regression and Support Vector Machines. To enrich the feature space with semantic information, Word2Vec embeddings are also explored as an alternative text representation. Additionally, a Deep Learning architecture based on learned word2vec embeddings and recurrent neural networks (LSTM) is employed to capture sequential and contextual dependencies within financial language.

The ultimate objective of this research is to develop a reliable classification framework capable of supporting decision-making processes in financial markets, where textual information plays a crucial role in shaping expectations and influencing investor behaviour. The ability to automatically interpret large volumes of this type of textual data can therefore provide valuable insights for analysts, institutions, and stakeholders.

A practical application of this project could be the integration of the sentiment classifier into financial monitoring systems, risk assessment tools, or algorithmic trading pipelines. By detecting shifts in sentiment in real time, such systems may help identify emerging trends, potential risks, or market reactions to corporate events.

The focus of this project lies not only in predictive performance, but also in interpretability and transparency. In financial contexts, model explainability is essential to ensure accountability and trust among stakeholders. For this reason, the research also considers Explainable Artificial Intelligence (XAI) techniques to better understand how textual features influence model predictions.

2. Dataset

In this research, we initially employed a dataset consisting of financial news articles related to Sri Lanka. The dataset contained approximately 12,000 observations. However, this dataset had two major problems:

- it didn't have enough observations for our research and for the techniques we wanted to implement.
- the class distribution was highly imbalanced, with a negligible number of neutral instances compared to positive and negative ones. This imbalance raised concerns regarding the model's ability to correctly learn and generalize neutral sentiment patterns.

We attempted to address these limitations by collecting more data and also find enough number of neutral instances to equilibrate the dataset.

We identified two additional datasets:

- one that contains the sentiments for financial news headlines from the perspective of a retail investor, related to the article: "Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts" published in Journal of the American Society for Information Science and Technology. It has 4837 observations.
- A large-scale dataset containing financial news from firms worldwide that has 76.772 observations.

However, upon merging these datasets we identified a potential source of bias in the first one: nearly all articles in this dataset explicitly mentioned "Sri Lanka." Since it had very few neutral instances, this posed the risk that the model could associate the term "Sri Lanka" with a specific sentiment (positive or negative), rather than learning sentiment from the broader contextual information.

We considered preprocessing strategies such as masking the country name. However, this approach proved ineffective. Since most observations in the original dataset referred to Sri Lanka, and the additional datasets under consideration contained significantly fewer country-specific mentions, replacing country names with a generic token (e.g., "country") would have introduced another bias. Specifically, the token "country" would still appear predominantly in positive and negative contexts, thus failing to resolve the original imbalance problem.

For these reasons, we decided to discard the Sri Lanka dataset and instead rely only on the alternative data sources. Therefore, the final dataset used in this research results from the combination of the two remaining sources, yielding a total of approximately 81.600 observations prior to any preparation and preprocessing step.

3. State of the Art

As we already said, this research on financial sentiment analysis has been carried out comparing classical machine learning approaches with deep learning architectures, using a variety of text representation techniques. Within the scientific literature, numerous similar

applications have been proposed, and some examples pertinent to this project are outlined below.

Malo et al. introduced the Financial PhraseBank, one of the dataset also used in this project, consisting of approximately 4,840 human-annotated sentences from financial news. Their work investigated how semantic orientations can be better detected in financial and economic texts by accommodating domain-specific language.

In a comparative study closely related to our methodology, Karanikola et al. evaluated classical machine learning models against deep learning architectures on financial sentiment data, testing both Bag-of-Words and TF-IDF representations combined with Logistic Regression and SVM. Their results showed that classical approaches tend to reach a performance ceiling, with Logistic Regression and TF-IDF among the best-performing traditional configurations, a finding consistent with our own results.

Also consistent with our findings, Thukral et al. observed that Logistic Regression on TF-IDF features achieved a validation accuracy of approximately 69% on a combined financial news dataset, a result comparable to what we obtained in our experiments with the same configuration.

Sharaff et al. conducted a study focused on financial news sentiment analysis combining Doc2Vec embeddings with LSTM, BiLSTM and stacked BiLSTM architectures. Their experiments found that the LSTM neural network performed best with an accuracy of 94%, though this result was achieved on a binary classification task and not on three classes like our research.

4. Methodology

The dataset used in this study is processed using different methodological approaches to financial sentiment classification, progressively moving from traditional frequency-based text representations to neural architectures capable of modelling semantic and sequential information.

4.1. Bag of words model

The Bag of Words model is perhaps one of the simplest yet most powerful techniques in natural language processing (NLP) to extract features from text documents.

The essence of this model is to convert text documents into vectors. Formally, given a **document vector space** $VS = \{W_1, W_2, \dots, W_n\}$, which contains all distinct words across the corpus, each document D can be represented as a vector:

$$D = \{Wd_1, Wd_2, \dots, Wd_n\}$$

where Wd_i is the weight of word W_i in document D, representing its frequency.

4.2. TF-IDF model

TF-IDF stands for Term Frequency-Inverse Document Frequency, a combination of two metrics: term frequency and inverse document frequency.

It can be represented as *tfidf=tf×idf,* where term frequency (tf) and inverse-document frequency (idf) represent the two metrics.

- **Term frequency** denoted by tf is what we had computed in the Bag of Words model:
$$tf\ (w, D)\ =\ f_{wD}$$
where f_{wD} denotes frequency for word w in document D

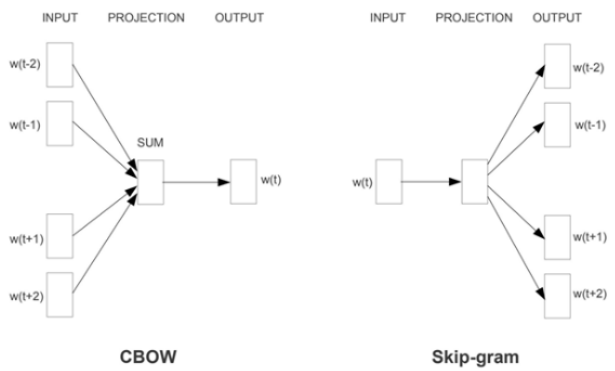
- **Inverse document frequency** is computed by dividing the total number of documents in our corpus by the number of documents having that term and then applying logarithmic scaling on the result:
$$idf(t) = 1 + log\frac{C}{1 + df(t)}$$
The logarithm helps scale the metric to dampen the effect of very frequent words.

- The **final TF-IDF feature vector** is obtained by multiplying TF and IDF for each word. Formally, the normalized TF-IDF vector is:
$$tfidf_{norm} = \frac{tfidf}{|tfidf|}$$
This normalization is necessary for managing word vectors of different size and for easy computation.

4.3. Word2Vec model

The Word2Vec model, released in 2013 by Google, is a neural-network-based approach that learns **distributed vector representations of words (word embeddings)** from very large text corpora using two main architectures:

- Continuous Bag of Words (CBOW)
- Skip-gram



- In the **CBOW model**, the distributed representations of context (or surrounding words) are combined to predict the word in the middle.
- While in the **Skip-gram model**, the distributed representation of the input/target word is used to predict the surrounding context words.

Internally, Word2Vec constructs a **vocabulary** based on the input text documents and learns vector representations for words based on CBOW and Skip gram. Once this is complete, it builds a model that can be used to extract word vectors for each word in a document.

The Word2Vec model has n dimensions, that means each word is represented as a n-element vector of real numbers, e.g.:

$$\text{Vector("market")} = [0.655, 0.219, 0.123, \dots, 0.002]$$

- Each word has n numerical features, which are coordinates along latent semantic axes.
- These dimensions are not explicitly interpretable (e.g., gender, plurality) but encode statistical co-occurrence patterns learned from the corpus (how often the word appears with others).
- Words with similar contexts have vectors close in space, capturing semantic and syntactic relationships.

Word2Vec offers several parameters that influence both training efficiency and model quality:

- **min_count**: This parameter controls the pruning of the internal vocabulary. Words that appear very rarely in the corpus are likely typos or noise, and there is insufficient data to train meaningful embeddings for them. Therefore, they can be safely ignored.

  For example:

  model = Word2Vec(sentences, min_count=10) *# default = 5*

  A reasonable range for min_count depends on the dataset size, typically between 0 and 100.

- **size**: This defines the dimensionality of the neural network layers (i.e., the word embeddings). Higher values require more data but can produce more accurate representations:

  model = Word2Vec(sentences, size=200) *# default = 100*

  Typical values range from tens to hundreds, depending on the dataset and task complexity.

- **workers**: This parameter enables parallelization to accelerate training:

  model = Word2Vec(sentences, workers=4) *# default = 1 (no parallelization)*

  Note that parallelization only works on multicore machines with Cython installed; otherwise, only a single core is used.

4.4. Logistic Regression

Logistic Regression is a probabilistic linear classifier widely used in text classification tasks. Given an input feature vector x, the model computes the probability of belonging to a class.

It is trained by minimizing the cross-entropy loss, which penalizes confident wrong predictions more heavily than uncertain ones, pushing the model to produce well-calibrated probability estimates.

In the context of text classification, the input feature vector typically corresponds to a BoW or TF-IDF representation of the document as we have done in this research.

4.5. Support Vector Machine

Support Vector Machines (SVM) are supervised learning models that seek to find the optimal hyperplane separating different classes in the feature space. The optimal hyperplane is defined as the one that maximizes the margin, that is, the distance between the hyperplane and the nearest data points of each class, known as support vectors. Intuitively, a larger margin corresponds to a more robust decision boundary and better generalization to unseen data.

When data is not linearly separable, SVMs employ the kernel trick to implicitly map input features into a higher-dimensional space where linear separation becomes feasible. In text classification, however, the linear kernel is often preferred due to the high dimensionality of the feature space and the typically sparse nature of BoW and TF-IDF representations.

For multiclass problems, such as the three-class sentiment classification in this research, the SVM is extended using a one-vs-rest (OvR) strategy, where one binary classifier is trained per class.

In this research, the SVM is applied in combination with both the Bag of Words and the TF-IDF representations.

4.6. LSTM

LSTMs are a special type of RNN, a class of neural networks designed to work with sequential data, that are designed specifically to handle long-term dependencies. This is achieved through **gates** to control what to store, forget, update, output.

Each LSTM unit maintains two internal states:

- a **short-term memory** h_t (**hidden state**),
- a **long-term memory** c_t (**cell state**).

The cell state enables information to persist across many time steps, allowing the network to retain relevant context over long sequences. The cell state is updated according to the following equation:

$$\mathrm{c_t} = \mathrm{f_t} * \mathrm{c_{t-1}} + \mathrm{i_t} * \mathrm{g_t}$$

Where:

$\mathrm{f_t}$= forget gate, which determines what portion of the previous cell state c_{t-1} should be retained. Its values range between 0 and 1: 0 means complete forgetting, 1 means complete retention.

$\mathrm{c_{t-1}}$= previous cell state

i_t= input gate, which controls how much new information should be stored.
g_t= candidate update, representing the new information that may be added to the cell state.

Therefore, the new **long- term memory (cell state)** is a combination of what we decide to keep from the past and what new information we decide to add.

This gated mechanism allows LSTMs to preserve relevant information over long time spans while discarding irrelevant content, effectively reducing the vanishing gradient problem.

Even if information is stored in long-term memory, it does not always need to be visible. The **output gate** decide what information should be revealed at each time step t.

$$\mathrm{h_t} = \mathrm{o_t} * \tanh(\mathrm{c_t})$$

Where:

- o_t is the output gate
- $\tanh(\mathrm{c_t})$ is the **hyperbolic tangent activation function** that keep values into the range [-1,1], ensuring numerical stability and introducing non-linearity.

4.7. Bidirectional LSTM

A Bidirectional LSTM (BiLSTM) extends the standard LSTM architecture by processing the input sequence in both forward and backward directions simultaneously. While a standard LSTM reads a sequence from left to right, only having access to past context at each time step, a BiLSTM runs two separate LSTM layers in parallel:

- a **forward LSTM**, which processes the sequence from the first to the last token,
- a **backward LSTM**, which processes the sequence from the last to the first token.

The hidden states produced by the two layers are then concatenated at each time step, producing a richer representation that captures both past and future context:

$$h_t = [h_t^{\{forward\}} ; h_t^{\{backward\}}]$$

This bidirectional reading is particularly beneficial for text classification tasks, where the sentiment or meaning of a word may depend not only on the words that precede it but also on those that follow.

For instance, in the sentence *"the stock, despite initial concerns, rose sharply"*, the word "concerns" can only be correctly interpreted as non-negative if the model has access to the word "rose" that comes after it. In financial sentiment analysis, where negations, contrastive conjunctions and hedging expressions are common, the BiLSTM's ability to capture full-sentence context makes it a more suitable architecture than a unidirectional LSTM.

5. Computational approach

5.1. Data preparation

After merging the datasets, a cleaning and validation phase was conducted in Excel. Duplicate sentences were initially removed to prevent redundancy and potential bias during model training. This operation led to a substantial reduction in the overall dataset size, resulting in a final total of 35.782 observations after duplicate removal. The integrity of sentiment labels was verified to ensure consistency across observations, and missing values were identified and excluded from the final corpus. Prior to balancing, the dataset was composed of 14,165 positive sentences, 8,247 negative sentences, and 13,370 neutral sentences.

The dataset was imported into Google Colab and stored as a Pandas DataFrame. An initial inspection was conducted through commands such as info(), columns(), and sample() to verify the dataset schema, confirm the presence of the key variables (Sentence and Sentiment), and check for structural anomalies.

A duplicate detection step was then performed to check and confirm the absence of duplicates, which were removed firstly with Excel. This ensured that each sentence appeared only once in the dataset, preventing repeated observations from biasing the training process.

The class distribution of the sentiment labels was subsequently examined using value_counts() and relative frequencies: the results highlighted an imbalance across categories, with certain sentiments more represented than others.

To address this issue, an under-sampling strategy was applied to the majority classes using the sample() function with a fixed random state, guaranteeing reproducibility. The dataset was then shuffled with sample(frac=1) to avoid ordering effects. The updated frequency distribution printed shows a more balanced representation of the three sentiment classes, with 8000 positive sentences, 8000 neutral sentences and the remaining 7638 negative sentences.

Last stage of data preparation was to divide the dataset into training, validation, and test sets using train_test_split() with the stratify parameter. This approach preserved the proportion of each class in all subsets. The outputs confirm that class distributions remain consistent across splits, providing a solid basis for model evaluation.

The prepared dataset was not only used for model training and evaluation but also served as the textual corpus for learning Word2Vec embeddings, ensuring consistency between the semantic representation and the classification pipeline

5.2. Data preprocessing

The preprocessing phase focused on cleaning, normalizing, and structuring textual data before transforming them into numerical inputs for the models. This step directly affects the quality of the features used by machine learning and deep learning models.

The process began with the text normalization: a text cleaning function was used to remove noise and normalize the input. The function applied ftfy.fix_text() to correct encoding inconsistencies,

while regular expressions (re.sub) were used to eliminate URLs, mentions, hashtags, and other non-informative patterns. Special characters, emoticons, redundant punctuation and whitespaces are also removed.

The preprocessing pipeline was partially differentiated depending on the type of model used in the project.

For traditional machine learning models, the cleaned text was prepared to support statistical feature extraction methods; the normalization process aimed to reduce textual variability while maintaining the lexical information necessary for vectorization techniques such as Bag-of-Words and TF-IDF.

The text was first converted to lowercase to ensure lexical consistency. Tokenization was then performed, segmenting each sentence into individual words, while non-alphabetic tokens were filtered out using isalpha(), removing numbers and residual symbols. Stop words were then eliminated; however, negation terms such as “no” and “not” were intentionally retained because of their strong influence on sentiment interpretation. Lemmatization was subsequently applied using WordNetLemmatizer(), reducing inflected forms to their base representation and decreasing vocabulary variability.

For the deep learning pipeline, preprocessing was designed to preserve the sequential structure of the text rather than aggressively simplifying it. The pipeline included lowercasing, tokenization, and filtering of non-alphabetic characters; both stop word removal and lemmatization were avoided intentionally. This choice was motivated not only by the requirements of sequential architectures such as LSTM, which rely on word order and contextual relationships, but also by the integration of Word2Vec embeddings. Word2Vec learns semantic representations from word co-occurrence patterns; therefore, excessive normalization (such as lemmatization or stop-word removal) could alter distributional properties and weaken the quality of learned embeddings.

Sentiment labels were prepared for computational use. The categorical classes (negative, neutral, positive) were mapped into numerical values through label encoding to ensure compatibility with machine learning algorithms and neural network training procedures. For deep learning purposes, one-hot encoding was applied, generating a matrix in which each label is represented by a binary vector.

The preprocessing phase guaranteed that textual data were standardized and correctly structured before being converted into numerical representations, supporting both traditional and deep learning modelling approaches.

5.3. Text representation

Following preprocessing, the textual data were transformed into numerical representations suitable for computational modelling. Since machine learning algorithms cannot directly process raw text, this phase converts sentences into structured numerical inputs that can be interpreted mathematically by the models.

In the script, different representation strategies were implemented depending on the modelling approach.

For traditional machine learning models, text was converted into numerical feature vectors using vectorization techniques available in Scikit-learn. In particular, Bag-of-Words and TF-IDF representations were generated through tools such as CountVectorizer and TfidfVectorizer. These methods transform each sentence into a numerical vector based on the presence and relevance of words within the corpus, producing matrices where rows correspond to sentences and columns correspond to vocabulary terms. These matrices were then used as input for classifiers such as Logistic Regression and Support Vector Machines implemented in the project.

In the first approach, Bag of Words, the vectorizer was fitted on the training set, building a vocabulary of terms and n-grams, and then applied to validation and test sets using. The use of ngram_range=(1,2) allowed the inclusion of both individual words and two-word combinations, capturing expressions relevant to sentiment analysis. The resulting matrices are sparse and high-dimensional, reflecting the size of the extracted vocabulary.

The second method TF-IDF extends the Bag of Words approach by weighting terms according to their importance in the corpus. Parameters such as min_df, max_df, and sublinear_tf were employed to filter noise and adjust term influence. The resulting matrices contain real-valued weights and provide a more informative representation of textual relevance.

The Word2Vec model was trained on the tokenized corpus to capture semantic relationships between financial terms. Each word was mapped to a dense vector in a continuous semantic space. Rather than being used solely to generate averaged sentence representations, the learned embeddings were employed to initialize the embedding layer of the Bidirectional LSTM architecture. This allowed the deep learning model to start from semantically meaningful word representations instead of random initialization, improving its ability to capture contextual dependencies in financial language.

For the deep learning component, a sequential representation strategy was adopted. For LSTM architecture, the textual corpus was processed using a tokenizer, which builds a vocabulary and assigns a unique integer index to each word. Each sentence was then converted into a sequence of integer indices and subsequently standardized in length applying the padding function, ensuring uniform input size across samples. This operation is necessary for batch processing and allows the Bidirectional LSTM architecture implemented in the project to handle textual inputs consistently.

To provide semantically meaningful input representations, Word2Vec embeddings were trained on the tokenized corpus and used to construct an embedding matrix aligned with the tokenizer vocabulary. Each word index was associated with its corresponding dense vector representation when available, while out-of-vocabulary words were initialized with small random values.

This embedding matrix was used to initialize the embedding layer of the Bidirectional LSTM model, allowing the network to start from semantically informed word representations rather

than random initialization. As a result, the deep learning architecture directly integrates distributional semantic information within the sequential modelling framework.

The text representation stage transforms textual information into numerical formats suitable for both traditional machine learning models and deep learning architectures. In particular, the integration of Word2Vec embeddings provided a semantically informed representation that enhanced the effectiveness of the Bidirectional LSTM model.

5.4. Training phase

After completing data preparation, preprocessing, and text representation, the training phase consisted of fitting each model on the training set and evaluating its generalization capability on the validation and test sets. Since the dataset had been previously balanced and split using stratified sampling, all models were trained under comparable and statistically coherent conditions.
The training process followed two parallel methodological paths: classical machine learning models trained on sparse vector representations (Bag-of-Words and TF-IDF), and a deep learning architecture trained on sequences initialized with Word2Vec embeddings.

Logistic Regression with Bag of Words
The first configuration involved Logistic Regression applied to the Bag-of-Words representation. After generating the BoW matrix, the classifier was trained and predicted using the meu.train_predict_model() function. In this setup, each sentence is represented as a high-dimensional sparse vector containing unigram and bigram frequencies (ngram_range=(1,2)), enabling the model to learn direct associations between lexical patterns and sentiment categories.

Logistic Regression optimizes a cross-entropy objective, producing probabilistic outputs for each class. During training, the model converged rapidly, reflecting the relatively linear separability of the data in the BoW feature space. The validation and test results reported in the final comparison table show an accuracy of 0.7197 and an F1 score of 0.7183, making this configuration the best-performing model in the project.

The strong performance of this configuration indicates that, in financial text, discriminative power is largely driven by the presence of specific lexical cues rather than by complex sequential dependencies.

Support Vector Machine with Bag of Words
The second configuration to training Bag of Word feature replaced Logistic Regression with a Support Vector Machine. Using the meu.train_predict_model() method of the SVM classifier, the model searched for the optimal separating hyperplanes in the feature space.
Unlike Logistic Regression, SVM maximizes the margin between classes. In practice, the performance achieved by SVM + BoW (accuracy 0.7171, F1 score 0.7163) was very close to Logistic

Regression, confirming that the decision boundary in the BoW space is largely linear and does not require more complex transformations.
The similarity in results between these two models suggests that the informational structure of the dataset can be effectively captured by linear decision functions.

Logistic Regression with TF-IDF

In the third configuration, Logistic Regression was trained on TF-IDF features. Unlike BoW, TF-IDF reweights terms according to their inverse document frequency, reducing the influence of very common words.
After fitting the vectorizer on the training set and transforming the validation and test sets, the classifier was again trained and predicted using meu.train_predict_model(). The resulting performance (accuracy 0.7033, F1 score 0.7022) was slightly lower than the BoW counterpart.
This outcome suggests that, in this specific financial domain, highly frequent technical terms still carry predictive information. Penalizing them through inverse document frequency weighting may reduce useful signal rather than enhance discriminative power.

Support Vector Machine with TF-IDF

The fourth configuration combined SVM with TF-IDF features. The training procedure mirrored the previous setups.
The resulting performance (accuracy 0.7042, F1 score 0.7028) remained consistent with Logistic Regression on TF-IDF but inferior to both BoW configurations. Again, this indicates that frequency-based lexical presence appears more informative than weighted frequency adjustments in this dataset.
Across all classical models, convergence was stable and did not exhibit overfitting patterns comparable to the neural architecture. The balanced nature of the dataset is reflected in the close alignment between accuracy, precision, recall, and F1 score reported in the comparison table.

Bidirectional LSTM with W2V

The deep learning training phase followed a substantially different paradigm from the classical models, based on a sequential learning approach combined with domain-relevant word representations. Sentences were first transformed into padded integer sequences using the Keras Tokenizer and pad sequences. The embedding layer was then initialized with weights derived from a Word2Vec model trained on the same corpus using a Skip-gram architecture, providing the network with semantically informed starting representations rather than random initialisation. The BiLSTM was compiled with categorical cross-entropy loss and the Adam optimizer, consistent with the one-hot encoded labels, and trained using *model.fit()* with validation data monitored at each epoch. Two callbacks were employed: *EarlyStopping*, to restore the best-performing weights based on validation loss, and *ReduceLROnPlateau*, to halve the learning rate when validation loss stagnated for three consecutive epochs. Training accuracy increased rapidly across epochs, exceeding 0.92 by epoch 6, while validation accuracy peaked at epoch 2 (~72%) and plateaued thereafter. Validation loss reached its minimum at epoch 2 and then rose steadily, exceeding 1.0 by the final epoch. This divergence is a clear indicator of overfitting: the model learned training-

specific patterns that do not fully generalize to unseen data. *EarlyStopping* correctly identified epoch 2 as the best checkpoint and restored those weights for final evaluation.

The first model configuration consisted of two Bidirectional LSTM layers. During training, a clear pattern emerged: training accuracy increased rapidly, while validation performance plateaued early and began to deteriorate after a few epochs. This behaviour indicates overfitting, meaning the model learned training-specific patterns that do not generalize to new data. Early stopping intervened early, signalling that the model reached its maximum generalization potential in the first training phases. To counter overfitting, we initially increased regularization via aggressive dropout (0.6). However, empirical results showed that excessive dropout does not necessarily improve generalization. On the contrary, excessive information loss during training makes the sequential representation unstable and reduces the model's ability to learn robust semantic patterns. This outcome aligns with recurrent network theory: when the linguistic signal is already limited, adding too much stochastic noise does not prevent overfitting but rather weakens learning. Based on this evidence, the architecture was simplified by adopting a single, more expressive Bidirectional LSTM with moderate regularization (0.2). The initial idea behind using two LSTM layers was to build a hierarchical text representation: the first layer would capture local word-level patterns, while the second would abstract a more complex semantic representation of the sentence. From a theoretical perspective, this is consistent with deep sequential models in natural language processing. However, the training behaviour observed, rapid growth in training accuracy with early validation plateau, showed that the second LSTM did not improve the representation but amplified noise, favouring overfitting. The architecture consists of a single Bidirectional LSTM of 64 units with moderate regularization (Dropout 0.3, recurrent_dropout 0.2). The single-layer configuration represents a rational trade-off between representational power and learning stability.

The *recurrent_dropout* parameter deserves specific mention: unlike standard dropout, which deactivates neurons between layers, recurrent dropout acts on the LSTM's internal recurrent connections across timesteps. This prevents the model from memorizing training-specific sequential patterns and encourages more robust temporal representations, particularly relevant for short financial sentences where individual word transitions are few and potentially noisy.

The key methodological insight from this experiment is twofold. First, for short financial sentences, sequential depth alone does not provide substantial benefits: the performance ceiling of a randomly initialized BiLSTM was comparable to classical bag-of-words models, suggesting that the bottleneck lies in the quality of the textual representation rather than model capacity. Second, pre-training word embeddings on the same corpus, even a moderately sized one, provides a sufficient initialisation advantage for the sequential model to outperform linear classifiers, confirming that the integration of representation learning and sequential modelling is more effective than either approach in isolation.

5.5. Long-Short Term Memory (LSTM) architecture

The application of the BiLSTM model to sentiment analysis was designed to evaluate how well a sequential architecture, combined with domain-relevant word representations, could capture

semantic information beyond bag-of-words or TF-IDF representations. The choice of a Bidirectional LSTM is motivated by its ability to model contextual dependencies in both forward and backward directions, a feature particularly useful in short texts where meaning depends on the interaction between words. The embedding layer was initialized with weights derived from a Word2Vec model trained on the same corpus, providing the network with semantically informed starting representations rather than random initialisation. The experimental pipeline was constructed following methodologically sound practices for deep learning on textual data. The dataset was first balanced through undersampling of majority classes, then split with stratification into training, validation, and test sets in a 70/15/15 ratio. This ensures that the model is evaluated on unseen data while maintaining a stable class distribution across splits. Text preprocessing included linguistic cleaning, tokenization, normalization, and conversion into numerical sequences using a tokenizer trained exclusively on the training set, thus avoiding data leakage. Sequences were padded to a fixed length to ensure homogeneity. As discussed in the previous paragraph, the training phase revealed a rapid increase in training accuracy accompanied by early stabilization and slight deterioration of validation performance, signalling overfitting in the more complex configuration. Consequently, simplifying the architecture to a single Bidirectional LSTM with moderate regularization led to more stable convergence, confirming that generalization was primarily constrained by data characteristics rather than model depth.

When looking at the application of LSTM to this dataset, it becomes clear how architecture and parameter choices affect not only final metrics but also model behaviour during training. Two main approaches were compared:

Double BiLSTM (32 units each) + Dropout 0.6

This model was relatively complex: two bidirectional LSTM layers with 32 units each, separated by high dropout (0.6) after each LSTM and after the intermediate Dense layer. From the first epochs, training accuracy increased rapidly (0.5876 at epoch 2, 0.7625 at epoch 3). However, validation accuracy remained around 0.6808 – 0.6983, and validation loss, starting at 0.7819, worsened from epoch 4 onward, reaching 1.5730. This indicates early overfitting: the model learned training patterns quickly but generalized poorly. Early stopping triggered at epoch 3, restoring weights from the epoch with minimum validation loss (0.7780).

Single BiLSTM (64 units) + Dropout 0.3

The second model simplified the architecture: a single bidirectional LSTM with 64 units and moderate dropout (0.3), plus *recurrent dropout* = 0.2 for internal regularization. This led to more stable behaviour: training accuracy at epoch 2 reached 0.7626 (like the first model), but validation loss reached a lower minimum of 0.7097 and validation accuracy of 0.6943. The simpler, moderately regularized structure allowed better generalization, with early stopping at epoch 2, preventing overfitting seen in the double LSTM.

Comparing the two models, key points emerge:

- The double LSTM has more parameters and aggressive dropout, causing rapid overfitting: high training accuracy (0.9508 at the end) but stagnant validation accuracy (0.688–0.698) and worsening validation loss.
- The single LSTM with 64 units and moderate dropout maintains a lower validation loss (0.7097) and comparable validation accuracy (0.6943), demonstrating greater stability and effective generalization while achieving similar training accuracy.

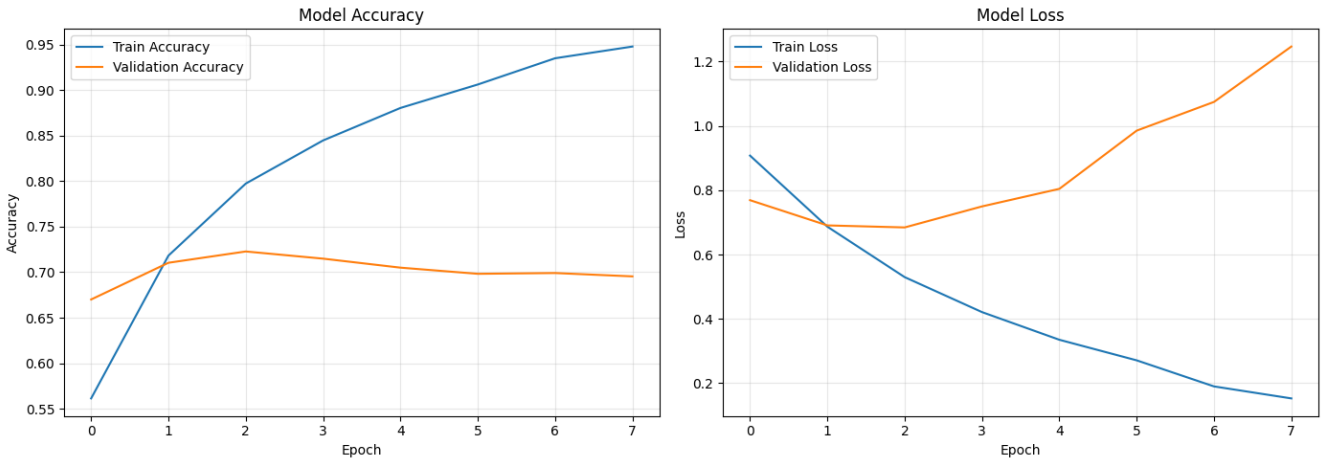
This shows that, for this dataset, the additional complexity of a double BiLSTM does not improve overall performance and instead increases overfitting risk. Conversely, a single, wider LSTM with moderate regularization achieves comparable accuracy with lower validation loss, demonstrating better generalization.

Recurrent Dropout

- Standard dropout "switches off" some connections in a layer to prevent overfitting.
- Recurrent dropout acts on the LSTM's internal states across timesteps, randomly ignoring a fraction of recurrent connections during training. In the single-layer BiLSTM, *recurrent_dropout* = 0.2 helped the model learn longer sequences without memorizing training-specific patterns, compensating for the lack of a second LSTM and improving generalization.

In the double BiLSTM with Dropout 0.6, adding recurrent dropout would likely have been counterproductive, excessively reducing capacity. In the single BiLSTM with Dropout 0.3, introducing *recurrent_dropout* = 0.2 improved stability, lowered validation loss, and maintained similar validation accuracy without increasing overfitting risk.

Accuracy and Loss (training vs validation)



The accuracy plot shows a rapid divergence between training and validation curves from epoch 1 onward: training accuracy climbs steeply across all 7 epochs, reaching 93%, while validation accuracy peaks at epoch 2 (72%) and then plateaus and slightly degrades, stabilising around 70% by the final epochs. This is a clear signature of overfitting, confirming that the model's capacity (2.1M parameters) exceeds what the dataset size and input sequence brevity can sustain.

The loss plot reinforces this picture with greater clarity: training loss decreases monotonically throughout all epochs, approaching 0.2 by epoch 6, while validation loss reaches its minimum at epoch 1 (0.68) and then rises steadily, exceeding 1.0 by the final epoch. The crossover between the two curves occurs around epoch 1-2, after which the divergence widens progressively — a textbook overfitting pattern.

EarlyStopping monitored validation loss with patience=5 and correctly identified epoch 2 as the best checkpoint, restoring those weights for final evaluation. This is why the test results (Accuracy=72.76%, F1=0.7276) reflect the model at its generalisation peak rather than at the end of training. The ReduceLROnPlateau callback triggered at epoch 4, halving the learning rate, but was insufficient to reverse the overfitting trend already well underway at that stage.

5.6. Models' outputs

```
==========================================================================
                    FINAL REPORT - MODELS COMPARISON
==========================================================================
                  Model  Accuracy  Precision  Recall  F1 Score
                 BiLSTM    0.7250     0.7273  0.7250    0.7255
              SVM + BoW    0.7239     0.7235  0.7239    0.7232
               LR + BoW    0.7197     0.7193  0.7197    0.7183
            SVM + TF-IDF    0.7050     0.7042  0.7050    0.7037
             LR + TF-IDF    0.7033     0.7024  0.7033    0.7022
==========================================================================

          Best model according to F1 Score: BiLSTM (0.7255)
```

BiLSTM

The BiLSTM achieves an F1 score of 0.7255, ranking first overall ahead of SVM + BoW (0.7239) and LR + BoW (0.7197). The per-class breakdown shows a balanced pattern: Neutral is the strongest class (F1: 0.75, Precision: 0.78), Negative follows closely (F1: 0.73, Recall: 0.7), while Positive remains the weakest class (F1: 0.70, Recall: 0.71), consistent with the pattern observed across all classical models, confirming that positive sentiment is the hardest to discriminate in this corpus regardless of the architecture. At the aggregate level, Precision and Recall are nearly identical (0.7282 vs 0.7276), indicating a well-balanced prediction behaviour: the model does not exhibit a systematic tendency to favour either precision or recall, unlike the classical models where the two metrics converge by construction due to the linear decision boundary. The training curve shows overfitting from epoch 3 onward, with training accuracy reaching 80% at epoch 2 while validation accuracy peaks at around 72% and degrades steadily thereafter. *EarlyStopping* correctly restored the epoch 2 weights, confirming that the model's capacity (2.1M parameters) remains disproportionate relative to the dataset size and the brevity of the input sequences.

SVM + BoW

SVM + BoW is the best-performing classical model (F1=0.7232), ranking second overall behind the BiLSTM (0.7255) and edging out LR + BoW by little in F1. The per-class distribution is nearly identical to LR + BoW: Neutral leads (F1=0.76, Recall=0.78), Negative is mid-range (F1=0.73), and Positive remains the hardest class (F1=0.68, Recall=0.67), confirming that the challenge lies in the representation rather than the classifier. The marginal advantage of SVM over LR may reflect the hinge loss's margin-maximisation objective, which can be slightly more robust on high-dimensional sparse feature spaces. However, the difference is too small to draw strong conclusions, and both models should be considered equivalent in practice on this dataset.

Logistic regression + BoW

Logistic Regression on BoW achieves an F1 score of 0.7183, ranking third overall behind BiLSTM (0.7255) and SVM + BoW (0.7232) by a negligible margin. Looking at the per-class breakdown, Neutral is the best-predicted class (F1=0.76, Recall=0.80): the model correctly identifies 960 out of 1,200 neutral sentences. Positive is the most difficult class (F1=0.68, Recall=0.65), with 203 positive sentences misclassified as negative, suggesting that mildly optimistic financial language is often lexically similar to negative or neutral phrasing. Negative sits in between (F1=0.72), with a balanced precision/recall trade-off. The L2 penalty distributes weight across the large vocabulary (141k features) without zeroing out rare but informative terms, which is particularly beneficial in a specialised financial corpus where low-frequency terms may still be strongly sentiment-bearing.

SVM + TF-IDF

SVM on TF-IDF achieves an F1 score of 0.7037, ranking fourth overall — marginally above LR + TF-IDF (Δ F1 = 0.0015) and approximately 2 points below SVM + BoW. The per-class structure is nearly identical to LR + TF-IDF: Neutral leads (F1: 0.75, Recall: 0.78), Negative is mid-range (F1: 0.70, Recall: 0.70), and Positive is the weakest class (F1: 0.66, Recall: 0.63), with 231 misclassifications — the single highest error count for the Positive class across all five models.

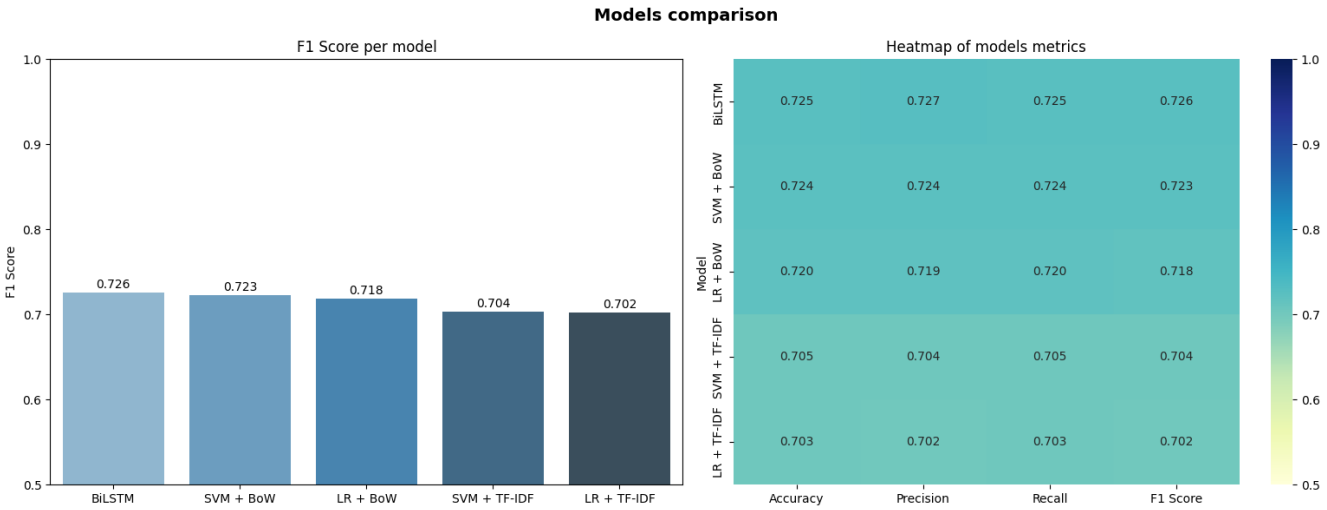
The result reinforces the conclusion already drawn from the BoW pair: the choice of representation has a greater impact on performance than the choice of classifier. The ~2 point F1 gap between the two SVM configurations (BoW vs TF-IDF) is consistent and stable, while the gap between SVM and LR on the same representation is negligible in both cases. TF-IDF's document-frequency penalization is systematically disadvantageous in this corpus, regardless of the classifier it is paired with.

Logistic regression + TF-IDF

Logistic Regression on TF-IDF achieves an F1 score of 0.7022, ranking last overall and sitting ~1.6 points below its BoW counterpart. The per-class breakdown mirrors the pattern observed across all models: Neutral is the strongest class (F1: 0.75, Recall: 0.78), Negative sits in the middle (F1: 0.70, Recall: 0.69), and Positive is again the weakest (F1: 0.66, Recall: 0.64), with 224 positive sentences misclassified — the highest misclassification count for that class across all models.

This result is counterintuitive with respect to the conventional preference for TF-IDF over raw counts, but consistent with the domain characteristics of the corpus: in highly specialised financial text, the absolute frequency of sentiment-bearing terms such as "earnings", "revenue", or "loss" is already informative on its own, and TF-IDF normalisation attenuates this signal by downweighting terms that appear frequently across documents. The more compact vocabulary (8k vs 142k features), produced by the min_df=5 threshold, may additionally discard discriminative low-frequency bigrams that BoW retains.

5.7. Models' comparison



The best model is BiLSTM with W2V-initialized Embedding (F1=0.7255), demonstrating that the integration of pre-trained word representations provides a measurable advantage over both random-initialisation deep learning and classical bag-of-words approaches. The two-tier structure observed among classical models is confirmed: both BoW-based models consistently outperform TF-IDF in F1, regardless of the classifier, indicating that the vectorisation choice matters more than the algorithm on this corpus.

Some relevant observations:

- Classical models based on Bag-of-Words remain highly competitive with deep learning approaches in this task, confirming that for short sentences and domain-specific language, sparse representations can be sufficient.
- TF-IDF does not improve performance over raw term counts in this corpus. The absolute frequency of financial terms appears to retain discriminative signal that normalization may partially attenuate.
- BiLSTM outperforms classical models once the embedding layer is seeded with domain-relevant W2V weights. This is consistent with the literature: on short sentences and specialised vocabulary, sequential models need a stronger initialisation signal to express

their advantage over bag-of-words approaches, which typically emerges with longer texts or more ambiguous language.

- Potential improvements include transfer learning with models pre-trained on financial corpora, such as FinBERT.

This outcome is consistent with several structural properties of the dataset and the classification task.

- First, financial text exhibits a highly standardized linguistic structure characterized by recurrent lexical patterns. In such domains, discriminative information is often conveyed by the presence or absence of specific technical terms rather than by their sequential configuration. The Bag-of-Words representation directly captures these lexical signals without introducing weighting transformations that may attenuate their relevance. The empirical underperformance of TF-IDF representations confirms that, in this domain, even highly frequent terms retain substantial predictive information and that document-frequency penalization results in a net loss of useful signal.
- Second, the W2V-initialized BiLSTM achieves the best overall F1 score (0.7276), outperforming all classical baselines. This result suggests that, on short financial sentences, sequential models require a strong initialisation signal to express their representational advantage: pre-trained word embeddings provide the necessary semantic foundation that the model would otherwise struggle to learn from a moderately sized dataset alone. The improvement is most pronounced on the Positive class — the hardest to discriminate across all models — where recall increased to 0.69, indicating that dense, context-aware representations are particularly beneficial where lexical ambiguity is highest.

Among classical models, the performance gap between linear classifiers is negligible, confirming that the choice of representation has a greater impact than the choice of classifier. Both BoW-based models consistently outperform their TF-IDF counterparts regardless of the algorithm used. An additional interpretative element derives from the balanced nature of the dataset, which leads to convergence among accuracy, precision, recall, and F1 score across all models. This convergence indicates the absence of systematic classification bias toward specific classes and strengthens the robustness of the comparison. In this setting, even small but consistent performance differences, such as the F1 gap between BiLSTM and SVM + BoW, become methodologically meaningful.

5.8. Predictions

Testing the BiLSTM model on new sentences provided in the script leads to better understand where the model performs better; it is highly reliable for strong signals, such as “strong earnings” (Positive 92%), “weak earnings” (Negative 93%), and “downgraded stock” (Negative 93%), hence BiLSTM learns well clear lexical patterns, explicit polarity and simple structures. Problems arise with financial ambiguity, particularly for neutral misclassification. The model links technical language to growth, showing a lack of pragmatic comprehension of the context; this is probably due to the medium-small size of the dataset ending up in a semantically poor neutrality.

The BiLSTM model struggles when an implicit economic interpretation is needed and sometimes where there is neutrality.

6. Explainability

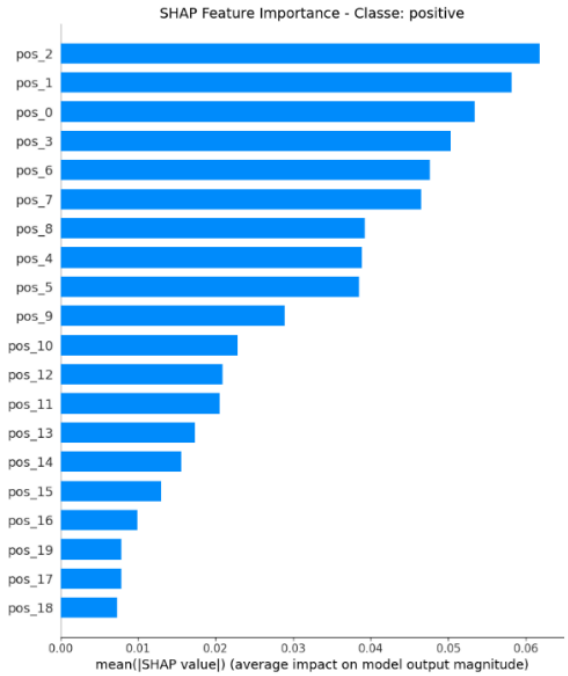
Explainability represents a crucial aspect in the field of Artificial Intelligence, particularly in the context of financial sentiment analysis, where model predictions may directly influence economic decisions. In financial environments, transparency is essential to ensure trust, accountability, and regulatory compliance. For this reason, integrating explainability techniques becomes fundamental.

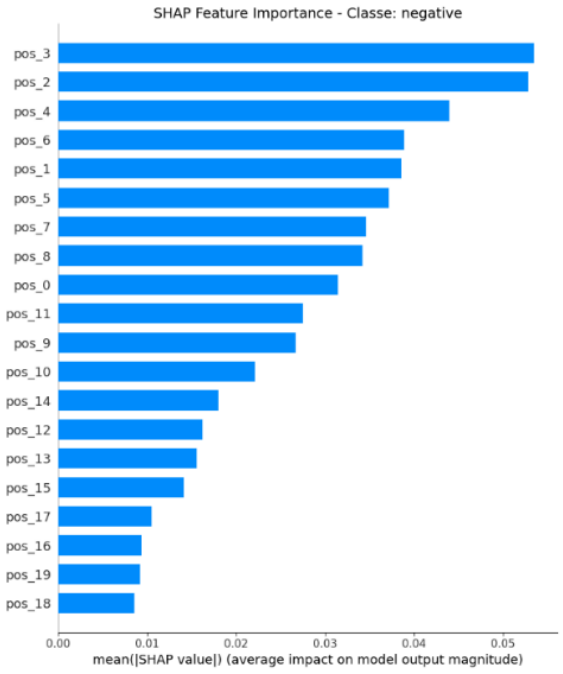
6.1 SHAP: SHapley Additive exPlanations

The selected approach for this project is Shapley Additive Explanations (SHAP). SHAP is one of the most widely used tools for understanding the predictions made by complex machine learning models. SHAP values provide insights into how each feature in a model contributes to its predictions.

In the context of this project, SHAP was applied specifically to the BiLSTM model for two main reasons. First, the BiLSTM achieved the highest overall performance among the deep learning architectures tested, making it the most relevant model to interpret. Second, the BiLSTM is inherently an opaque model. its internal representations are distributed across thousands of parameters and hidden states, making it impossible to directly inspect how individual input features influence predictions. This opacity makes post-hoc explainability techniques not just useful, but necessary. The classical models such as Logistic Regression and SVM, by contrast, are more decomposable by nature, as their learned weights can be more directly associated with individual lexical features.

6.1.1 Global Interpretability: Summary Plots by Token Position





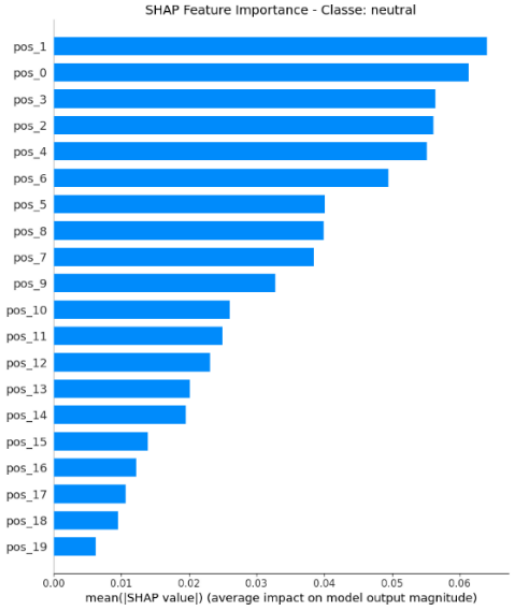


Figure 1 – SHAP Feature Importance by Token Position: Positive (left), Neutral (centre), Negative (right)

The three summary plots above display the mean absolute SHAP values aggregated by token position (pos_0 through pos_19) across 50 test samples. Since the model operates on padded sequences, early positions (pos_0–pos_8) consistently dominate across all three classes, suggesting that the model relies heavily on the beginning of each sentence to form its predictions. This pattern is consistent with financial news headlines, which typically front-load the most semantically relevant information.

The magnitude of SHAP values is broadly similar across the three classes

6.1.2 Local Interpretability: Single Sentence Analysis

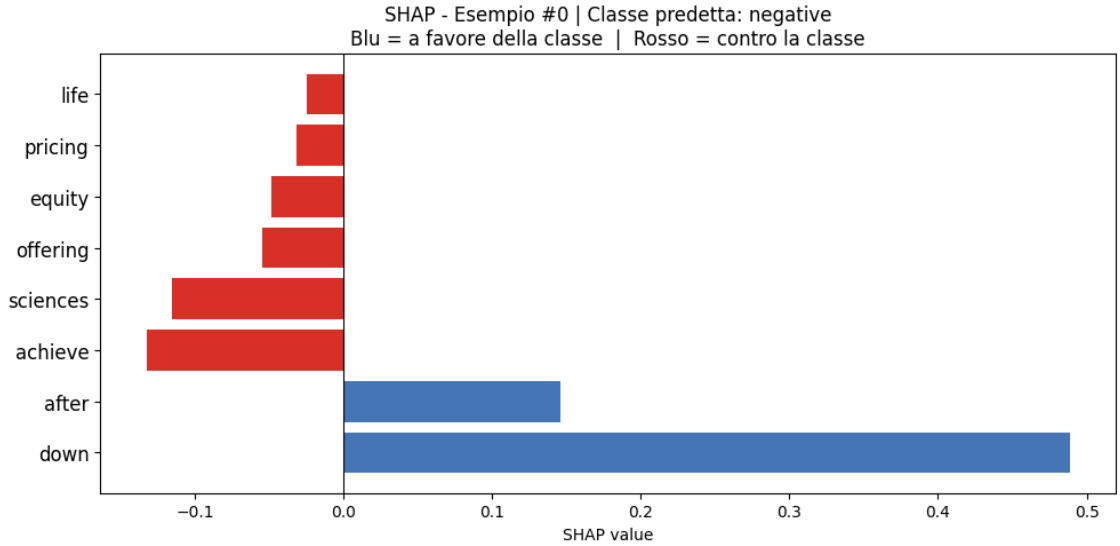


Figure 2 – SHAP barplot for an example phrase | Predicted class: negative

The barplot above analyses the sentence "achieve life sciences down after pricing equity offering", correctly classified as negative with 46.1% confidence. Blue bars indicate features pushing the prediction toward the negative class; red bars indicate features pushing away from it.

"Down" is by far the strongest driver toward the negative class (SHAP ≈ +0.49), followed by "after" (≈ +0.15). Both terms act as bearish signals in financial language: "down" directly implies a price decline, while "after" contextualises the drop because of a corporate event (equity offering). Conversely, words like "achieve", "sciences", "offering", "equity", "pricing" and "life" push the prediction away from negative, likely because they are associated with neutral or positive corporate announcements in the training set.

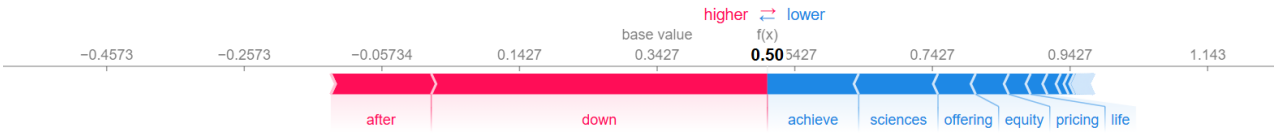


Figure 3 – Force plot for an example phrase| Predicted class: negative

The force plot provides an alternative visualisation of the same prediction, showing how features collectively push the output from the baseline value (0.34) that is, the average predicted probability for the negative class across all background samples, toward the final prediction (0.50). The width of each segment is proportional to the magnitude of its SHAP.

6.1.3 Global Word-Level Analysis: Top Words per Class

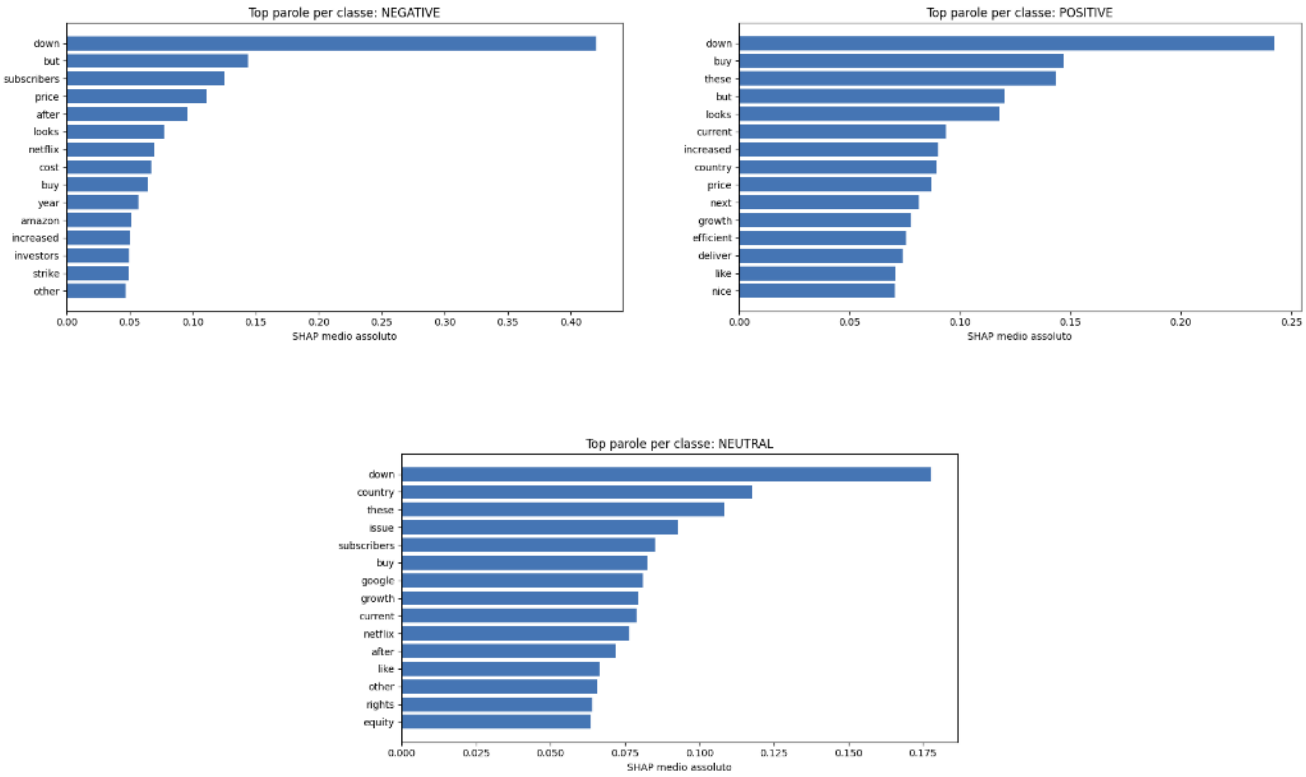


Figure 4 – Top 15 words by mean absolute SHAP value: Negative (left), Neutral (below), Positive (right)

The aggregated word-level SHAP analysis shows the top 15 most influential words per class, computed as the mean absolute SHAP value over 50 test samples (minimum 2 occurrences per word). This view is more linguistically interpretable than the positional summary plots and directly surfaces the lexical patterns learned by the model.

For the Negative class, "down" (0.420) dominates by a large margin — nearly three times more impactful than the second word "but" (0.144). This aligns with financial intuition: "down" is an unambiguous bearish signal. Other top negative words include "subscribers", "price", "after", "cost", "strike", "amazon" and "netflix", suggesting the model associates declining subscription metrics and streaming industry tension with negativity.

For the Neutral class, "down" again leads (0.178), but with a much smaller margin over the other words, suggesting it contributes informationally without being decisive. Contextual and structural words like "country", "these", "issue", "rights" and "equity" are prominent, consistent with neutral sentences describing factual situations without strong directional bias.

For the Positive class, "down" unexpectedly tops the ranking (0.242), followed by "buy" (0.147), "these" (0.143) and "but" (0.120). The presence of "down" here can be explained by contrastive constructions such as "costs are down, but profits are up" or "down from highs but still a buy", contexts where "down" does not carry a negative final sentiment. Words like "buy", "growth", "efficient", "deliver" and "nice" are clearly aligned with positive sentiment in financial text.

The most notable cross-class finding is the dominance of "down" across all three classes. Rather than functioning as a clean negative-only signal, the model has learned that "down" is a high-information word whose sentiment depends entirely on context. This behaviour is linguistically sound but also explains some of the model's misclassifications in ambiguous sentences, and highlights the inherent difficulty of financial sentiment classification compared to general-domain tasks.

In financial applications, explainability is not merely a technical add-on but a necessary component. Stakeholders — including investors, analysts, and regulatory bodies — require understandable and justifiable predictions. By incorporating SHAP and complementary interpretability techniques, this project ensures that model decisions remain transparent, interpretable, and aligned with ethical AI principles.

7.Ethics of AI

Financial sentiment analysis has direct implications for economic decisions, which makes ethical reflection a necessary part of the design process. This section discusses the main ethical issues raised by our project and how we addressed them.

7.1. Fairness Across Sentiment Classes

Our results consistently show that positive sentiment is the hardest class to classify correctly across all models. Positive financial language tends to be hedged, indirect, and lexically similar to neutral statements, making it structurally more ambiguous. This asymmetry matters in practice: a classifier deployed in a risk monitoring or trading system that systematically underestimates positive signals will produce skewed outputs with real financial consequences. This potential problem should be declared openly to any user of the model.

7.2. Transparency and Explainability

The integration of SHAP into this project is not simply a technical add-on: it is an ethical commitment to transparency. Financial decision-making environments whether in risk assessment, investment analysis, or algorithmic trading operate under regulatory frameworks that increasingly demand explainability as a condition of deployment. A model that produces a sentiment label without any accessible rationale is, from a governance standpoint, a black box that cannot be audited, contested, or trusted.

7.3.Responsibility in Deployment.

The outputs of this classifier can influence real financial decisions. Therefore, it should be positioned as a support tool for human analysts, not as an autonomous decision-maker. Consistent with the principle of meaningful human control, any real-world integration of a system like this must preserve the analyst's ability to review, override, and contest model outputs.

7.4.Privacy and Data Provenance

The datasets used in this project are publicly available and consist of financial news text, a form of public information by design and we don't have privacy concerns. However, when financial sentiment classifiers are extended to social media data, corporate communications, or other alternative data sources, the privacy can raise questions about data governance and the boundaries of legitimate inference. This is a limitation to keep in mind for any extension of the current approach.

8. **Working demo**

9. Conclusion

This project explored the application of both classical machine learning and deep learning techniques to financial sentiment analysis, with the aim of identifying how different textual representations and modelling approaches influence classification performance in a specialised economic domain.

The results confirm that financial sentiment classification is strongly dependent on the representation of textual information. Across all experiments, Bag-of-Words consistently outperformed TF-IDF, regardless of the classifier employed. This outcome suggests that, in financial language, the raw frequency of domain-specific terms retains significant predictive value. Penalising frequent words through TF-IDF weighting may reduce relevant signals rather than enhance model performance, especially in highly structured and technical corpora.

Among the tested models, the Bidirectional LSTM initialised with Word2Vec embeddings achieved the best overall performance, with the highest F1 score. This result highlights the importance of combining sequential modelling with semantically informed word representations. The integration of Word2Vec provided the neural architecture with a meaningful starting point, allowing it to capture contextual dependencies that are not accessible to purely frequency-based approaches.

However, the improvement over classical models remained moderate, indicating that for short and domain-specific financial texts, simpler linear models remain highly competitive.

The experiments also revealed important methodological insights. Increasing model complexity, such as adopting a double BiLSTM architecture, did not lead to performance improvements and instead amplified overfitting. A simpler configuration, based on a single Bidirectional LSTM with moderate regularisation, proved more stable and achieved better generalisation. This finding emphasises the importance of balancing model capacity with dataset size and linguistic characteristics.

Another relevant outcome concerns the intrinsic difficulty of financial sentiment analysis. Financial language is often ambiguous and context-dependent, and sentiment is rarely conveyed by isolated words alone. Terms such as “down” may appear in positive, negative, or neutral contexts depending on the surrounding information, making classification particularly challenging. The explainability analysis confirmed that the models capture meaningful lexical and contextual patterns but also highlighted limitations in handling neutral statements and implicit economic interpretations.

From a practical perspective, the findings suggest that classical models based on Bag-of-Words remain strong baselines for financial sentiment tasks, especially when dealing with short texts such as headlines or brief news summaries. Deep learning architectures provide measurable benefits when combined with domain-specific embeddings, but their superiority is not overwhelming in moderately sized datasets. Consequently, model selection in real-world applications should consider not only predictive performance but also interpretability, computational cost, and ease of implementation.

Future research could extend this work by incorporating transformer-based architectures and transfer learning techniques using models pre-trained on financial corpora, such as FinBERT. Expanding the dataset and improving the representation of neutral sentiment could also enhance model robustness. Additionally, further development of explainable AI techniques would support the adoption of sentiment analysis tools in financial decision-making environments, where transparency and accountability are essential.

This project demonstrates that financial sentiment analysis benefits from an integrated approach combining lexical representations, sequential modelling, and explainability techniques. While deep learning provides incremental improvements, domain-aware text representations remain a key determinant of performance. These findings reinforce the idea that, in specialised financial contexts, the effectiveness of a model depends not only on the algorithm adopted, but also on the way linguistic information is structured, processed, and interpreted.

10. Bibliography

dilkasithari-IT/news_data · Datasets at Hugging Face

RobertoMCA97/financial_sentiment_analysis_train_compilation_v2 · Datasets at Hugging Face

Sentiment Analysis for Financial News

[1] Malo et al. (2014). Good Debt or Bad Debt: Detecting Semantic Orientations in Economic Texts. *JASIST*, 65(4), 782–796.

[2] Karanikola et al. (2023). Financial Sentiment Analysis: Classic Methods vs. Deep Learning Models. *Intelligent Decision Technologies*, 17(4), 893–915.

[3] Thukral et al. (2025). Financial Sentiment Analysis and Classification: A Comparative Study. *JRFM*, 13(2), 75.

[5] Sharaff, A., Chowdhury, T.R. & Bhandarkar, S. (2023). LSTM based Sentiment Analysis of Financial News. *SN Computer Science*, 4, 584.